Assignment 2:
Joint-Space Trajectory Generation and Smoothness Analysis

Jaypal

Objective

The objective of this assignment is to study how a robotic arm moves between two joint configurations over time. Instead of considering only static poses, continuous joint-space trajectories are generated and analyzed for smoothness.

Problem Description

Consider a 2-link planar robotic arm with joint angles:

$$(q_1^{start}, q_2^{start}) \rightarrow (q_1^{end}, q_2^{end})$$

The robot must move between these configurations over a fixed time duration T. The joint trajectories are represented as:

$$q_1(t),\ q_2(t), \quad t \in [0, T]$$

Two trajectory types are generated:

- Linear joint-space trajectory
- Smooth polynomial (cubic) joint-space trajectory

Linear Joint-Space Trajectory

In a linear trajectory, each joint angle changes linearly with time:

$$q_i(t) = q_i^{start} + \frac{q_i^{end} - q_i^{start}}{T} t$$

This method is simple but introduces sudden changes in velocity at the start and end of motion.

Python Code: Linear Trajectory

```
import numpy as np
import matplotlib.pyplot as plt

T = 5
t = np.linspace(0, T, 100)

q1_start, q2_start = 0, 0
q1_end, q2_end = np.pi/2, np.pi/4

q1_linear = q1_start + (q1_end - q1_start) * t / T
q2_linear = q2_start + (q2_end - q2_start) * t / T

plt.figure()
plt.plot(t, q1_linear, label='q1 Linear')
plt.plot(t, q2_linear, label='q2 Linear')
plt.xlabel('Time (s)')
plt.ylabel('Joint Angle (rad)')
plt.title('Linear Joint-Space Trajectory')
plt.legend()
plt.grid()
plt.show()
```

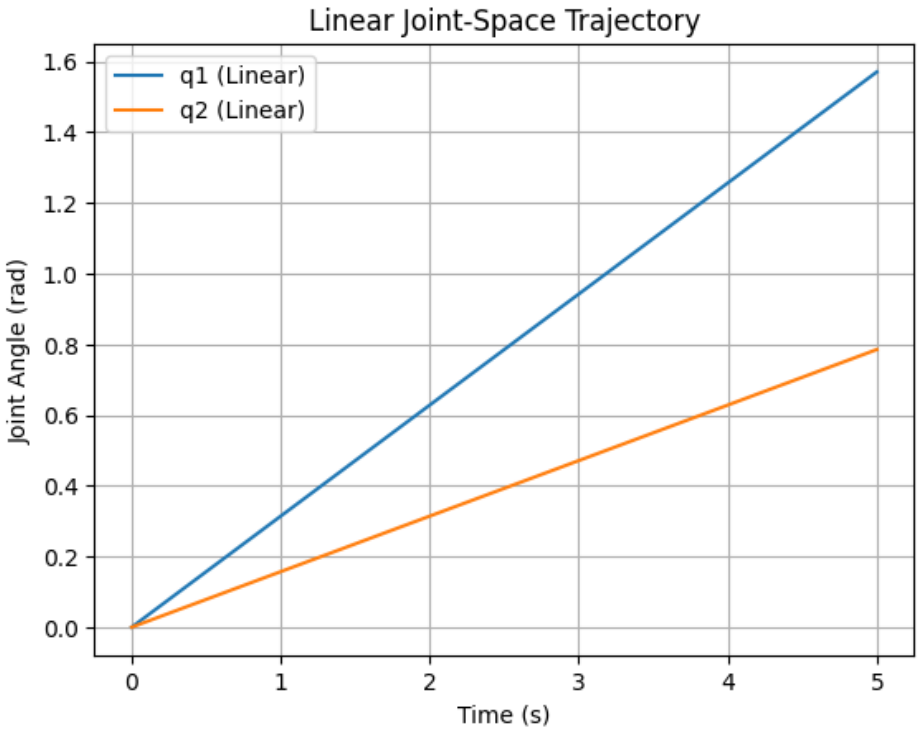

Figure 1: Linear joint-space trajectory for joints q_1 and q_2

Smooth Polynomial (Cubic) Trajectory

To ensure smooth motion, a cubic polynomial trajectory is used:

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

Boundary conditions:

$$q(0) = q_{start}, \quad q(T) = q_{end}$$
$$\dot{q}(0) = 0, \quad \dot{q}(T) = 0$$

Solving these conditions gives:

$$a_0 = q_{start}, \quad a_1 = 0$$
$$a_2 = \frac{3(q_{end} - q_{start})}{T^2}, \quad a_3 = \frac{-2(q_{end} - q_{start})}{T^3}$$

Python Code: Cubic Trajectory

```
def cubic_trajectory(q_start, q_end, T, t):
    a0 = q_start
    a1 = 0
    a2 = 3*(q_end - q_start)/T**2
    a3 = -2*(q_end - q_start)/T**3
    return a0 + a1*t + a2*t**2 + a3*t**3

q1_cubic = cubic_trajectory(q1_start, q1_end, T, t)
q2_cubic = cubic_trajectory(q2_start, q2_end, T, t)

plt.figure()
plt.plot(t, q1_cubic, label='q1 Cubic')
plt.plot(t, q2_cubic, label='q2 Cubic')
plt.xlabel('Time (s)')
plt.ylabel('Joint Angle (rad)')
plt.title('Smooth Polynomial Joint-Space Trajectory')
plt.legend()
plt.grid()
plt.show()
```

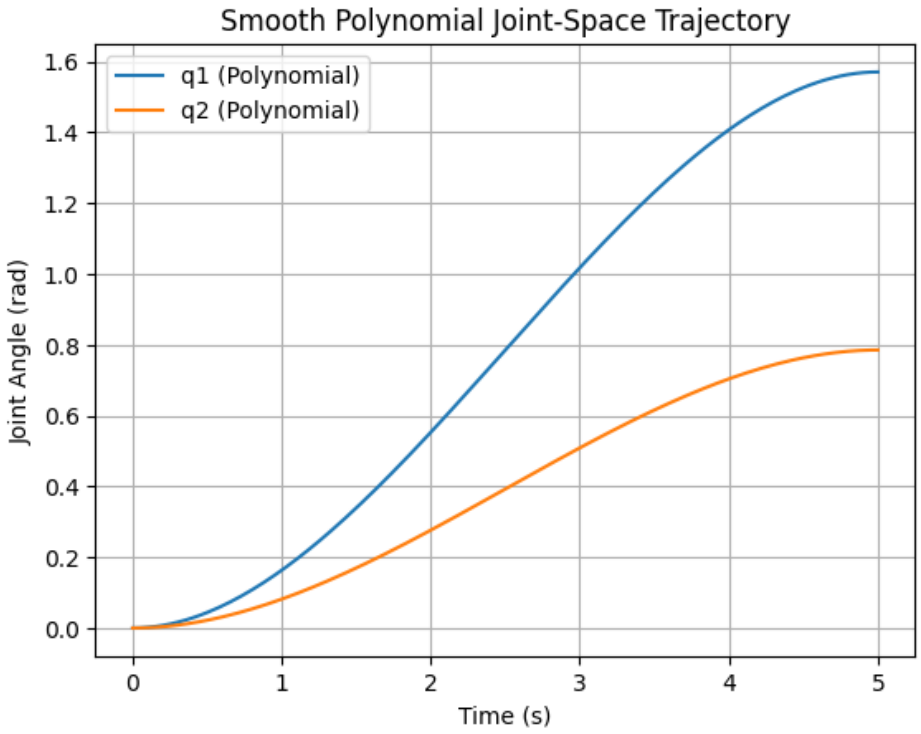

Figure 2: Smooth cubic joint-space trajectory for joints q_1 and q_2

Analysis and Comparison

The linear trajectory exhibits abrupt velocity changes at the start and end of motion. In contrast, the cubic trajectory ensures zero velocity at the boundaries, resulting in smoother motion.

Python Code: Trajectory Comparison

```
plt.figure()
plt.plot(t, q1_linear, '--', label="q1 Linear")
plt.plot(t, q1_cubic, label="q1 Cubic")
plt.plot(t, q2_linear, '--', label="q2 Linear")
plt.plot(t, q2_cubic, label="q2 Cubic")
plt.xlabel("Time (s)")
plt.ylabel("Joint Angle (rad)")
plt.title("Comparison of Linear and Polynomial Trajectories")
plt.grid()
plt.legend()
plt.show()
```

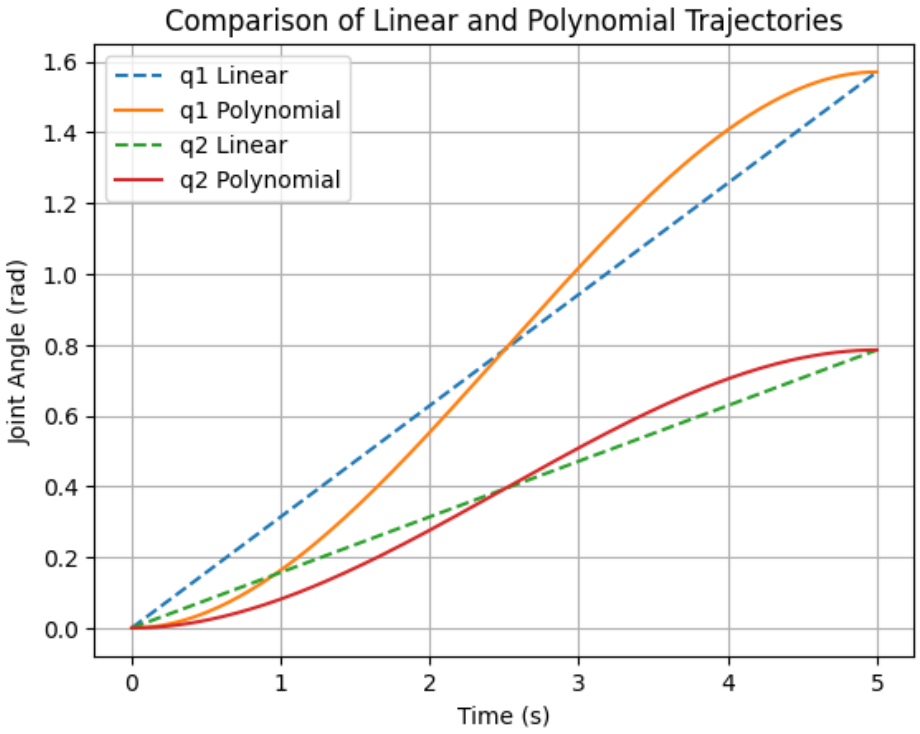

Figure 3: Comparison of linear and cubic polynomial trajectories

Conclusion

Linear joint-space trajectories are simple to implement but produce abrupt velocity changes at the start and end of motion. These sudden changes can cause jerks, vibrations, and increased wear in robotic joints. Smooth polynomial trajectories ensure zero velocity at 3 both the beginning and end, resulting in gentler motion. This smoothness is essential for accurate control, safety, and longer hardware life. Polynomial trajectories are therefore more suitable for real robotic systems. They are widely used in industrial robots and motion planning applications. .